

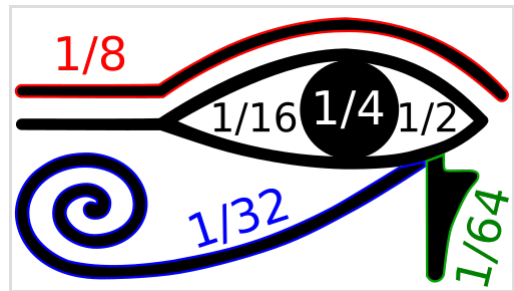
A **binary number** is a number expressed in the **base-2 numeral system** or **binary numeral system**, a method for representing numbers that uses only two symbols for the natural numbers: typically 0 (zero) and 1 (one). A *binary number* may also refer to a rational number that has a finite representation in the binary numeral system, that is, the quotient of an integer by a power of two.

The base-2 numeral system is a positional notation with a radix of 2. Each digit is referred to as a bit, or binary digit. Because of its straightforward implementation in digital electronic circuitry using logic gates, the binary system is used by almost all modern computers and computer-based devices, as a preferred system of use, over various other human techniques of communication, because of the simplicity of the language and the noise immunity in physical implementation.^[1]

History

The modern binary number system was first studied in Europe in the 16th and 17th centuries by Thomas Harriot, and decades later by Gottfried Leibniz, who is credited for the invention.^[2] However, systems related to binary numbers have appeared earlier in multiple cultures including ancient Egypt, China, Europe and India, e.g. in relation to divination using binary lots.^[3]

Egypt



Arithmetic values thought to have been represented by parts of the Eye of Horus

The scribes of ancient Egypt used two different systems for their fractions, Egyptian fractions (not related to the binary number system) and Horus-Eye fractions (so called because some historians of mathematics believed that the symbols used for this system could be arranged to form the eye of Horus, although this has been disputed).^[4] Horus-Eye fractions are a binary numbering system for fractional quantities of grain, liquids, or other measures, in which a fraction of a hekat is expressed as a sum of the binary fractions $\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{8}$, $\frac{1}{16}$, $\frac{1}{32}$, and $\frac{1}{64}$. Early forms of this system can be found in documents from the Fifth Dynasty of Egypt, approximately 2400 BC, and its fully developed hieroglyphic form dates to the Nineteenth Dynasty of Egypt, approximately 1200 BC.^[5]

Decimal number	Binary number
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

The method used for ancient Egyptian multiplication is also closely related to binary numbers. In this method, multiplying one number by a second is performed by a sequence of steps in which a value (initially the first of the two numbers) is either doubled or has the first number added back into it; the order in which these steps are to be performed is given by the binary representation of the second number. This method can be seen in use, for instance, in the Rhind Mathematical Papyrus, which dates to around 1650 BC.^[6]

China

The *I Ching* dates from the 9th century BC in China.^[7] The binary notation in the *I Ching* is used to interpret its quaternary technique of divination.^[8]

It is based on taoistic duality of yin and yang.^[9] Eight trigrams (Bagua) and a set of 64 hexagrams ("sixty-four" gua), analogous to the three-bit and six-bit binary numerals, were in use at least as early as the Zhou dynasty of ancient China.^[7]

The Song dynasty scholar Shao Yong (1011–1077) rearranged the hexagrams in a format that resembles modern binary numbers, although he did not intend his arrangement to be used mathematically.^[8] Viewing the least significant bit on top of single hexagrams in Shao Yong's square^[10] and reading along rows either from bottom right to top left with solid lines as 0 and broken lines as 1 or from top left to bottom right with solid lines as 1 and broken lines as 0 hexagrams can be interpreted as sequence from 0 to 63. ^[11]



Daoist Bagua

Classical antiquity

Etruscans divided the outer edge of divination livers into sixteen parts, each inscribed with the name of a divinity and its region of the sky. Each liver region produced a binary reading which was combined into a final binary for divination.^[12]

Divination at Ancient Greek Dodona oracle worked by drawing from separate jars, questions tablets and "yes" and "no" pellets. The result was then combined to make a final prophecy.^[13]

India

The Indian scholar Pingala (c. 2nd century BC) developed a binary system for describing prosody.^{[14][15]} He described meters in the form of short and long syllables (the latter equal in length to two short syllables).^[16] They were known as *laghu* (light) and *guru* (heavy) syllables.

Pingala's Hindu classic titled Chandaḥśāstra (8.23) describes the formation of a matrix in order to give a unique value to each meter. "Chandaḥśāstra" literally translates to *science of meters* in Sanskrit. The binary representations in Pingala's system increases towards the right, and not to

the left like in the binary numbers of the modern positional notation.^[17] In Pingala's system, the numbers start from number one, and not zero. Four short syllables "0000" is the first pattern and corresponds to the value one. The numerical value is obtained by adding one to the sum of place values.^[18]

West Africa

The Ifá is a West African divination system popular among the Yoruba tribe of the Old Oyo Empire. Similar to the *I Ching*, but has up to 256 binary signs,^[19] unlike the *I Ching* which has 64. The number comes from squaring 16 which also matches the total possibilities in an 8-bit sequence. In Ifá divination, this reflects the possible outcomes called *Odú* (<https://blogs.lse.ac.uk/impactofsocialsciences/2025/08/22/what-ifa-an-indigenous-binary-knowledge-system-can-teach-us-about-ai/>). These *Odú* are determined using an *Opẹlẹ* chain, which has 8 seeds. Each seed can land in one of two positions (open or closed) creating all the possible combinations. The Ifá originated in 15th century West Africa among Yoruba people. In 2008, UNESCO added Ifá to its list of the "Masterpieces of the Oral and Intangible Heritage of Humanity".^{[20][21]}

Other cultures

The residents of the island of Mangareva in French Polynesia were using a hybrid binary-decimal system before 1450.^[22] Slit drums with binary tones are used to encode messages across Africa and Asia.^[9] Sets of binary combinations similar to the *I Ching* have also been used in traditional African divination systems, such as Ifá among others, as well as in medieval Western geomancy. The majority of Indigenous Australian languages use a base-2 system.^[23]

Western predecessors to Leibniz

In the late 13th century Ramon Llull had the ambition to account for all wisdom in every branch of human knowledge of the time. For that purpose he developed a general method or "Ars generalis" based on binary combinations of a number of simple basic principles or categories, for which he has been considered a predecessor of computing science and artificial intelligence.^[24]

In 1605, Francis Bacon discussed a system whereby letters of the alphabet could be reduced to sequences of binary digits, which could then be encoded as scarcely visible variations in the font in any random text.^[25] Importantly for the general theory of binary encoding, he added that this method could be used with any objects at all: "provided those objects be capable of a twofold difference only; as by Bells, by Trumpets, by Lights and Torches, by the report of Muskets, and any instruments of like nature".^[25]

In 1617, John Napier described a system he called location arithmetic for doing binary calculations using a non-positional representation by letters. Thomas Harriot investigated several positional numbering systems, including binary, but did not publish his results; they were found later among his papers.^[26] Possibly the first publication of the system in Europe was by Juan Caramuel y Lobkowitz, in 1700.^[27]

Leibniz

Leibniz wrote in excess of a hundred manuscripts on binary, most of them remaining unpublished.^[28] Before his first dedicated work in 1679, numerous manuscripts feature early attempts to explore binary concepts, including tables of numbers and basic calculations, often scribbled in the margins of works unrelated to mathematics.^[28]

His first known work on binary, "*On the Binary Progression*", in 1679, Leibniz introduced conversion between decimal and binary, along with algorithms for performing basic arithmetic operations such as addition, subtraction, multiplication, and division using binary numbers. He also developed a form of binary algebra to calculate the square of a six-digit number and to extract square roots.^[28]



Gottfried Leibniz

His most well known work appears in his article *Explication de l'Arithmétique Binaire* (published in 1703). The full title of Leibniz's article is translated into English as the "*Explanation of Binary Arithmetic, which uses only the characters 1 and 0, with some remarks on its usefulness, and on the light it throws on the ancient Chinese figures of Fu Xi*".^[29] Leibniz's system uses 0 and 1, like the modern binary numeral system. An example of Leibniz's binary numeral system is as follows:^[29]

0 0 0 1	numerical value 2^0
0 0 1 0	numerical value 2^1
0 1 0 0	numerical value 2^2
1 0 0 0	numerical value 2^3

While corresponding with the Jesuit priest Joachim Bouvet in 1700, who had made himself an expert on the *I Ching* while a missionary in China, Leibniz explained his binary notation, and Bouvet demonstrated in his 1701 letters that the *I Ching* was an independent, parallel invention of binary notation. Leibniz and Bouvet concluded that this mapping was evidence of major Chinese accomplishments in the sort of philosophical mathematics he admired.^[30] Of this parallel invention, Leibniz wrote in his "Explanation Of Binary Arithmetic" that "this restitution of their meaning, after such a great interval of time, will seem all the more curious."^[31]

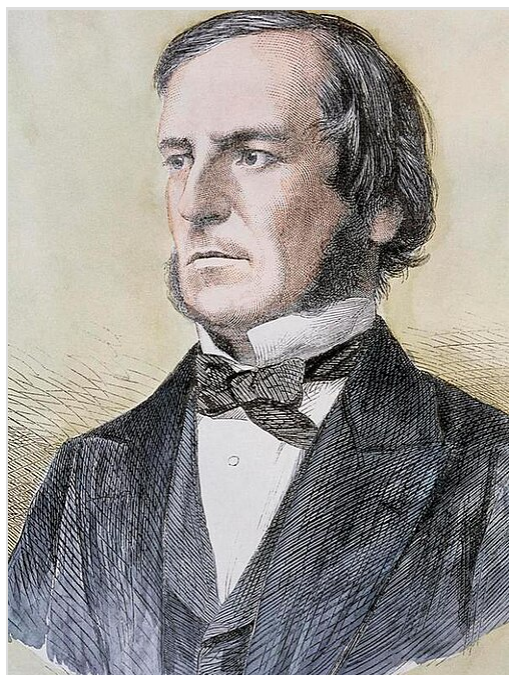
The relation was a central idea to his universal concept of a language or characteristica universalis, a popular idea that would be followed closely by his successors such as Gottlob Frege and George Boole in forming modern symbolic logic.^[32] Leibniz was first introduced to the *I Ching* through his contact with the French Jesuit Joachim Bouvet, who visited China in 1685 as a missionary. Leibniz saw the *I Ching* hexagrams as an affirmation of the universality of his own religious beliefs as a Christian.^[33] Binary numerals were central to Leibniz's theology. He believed that binary numbers were symbolic of the Christian idea of creatio ex nihilo or creation out of nothing.^[34]

[A concept that] is not easy to impart to the pagans, is the creation *ex nihilo* through

God's almighty power. Now one can say that nothing in the world can better present and demonstrate this power than the origin of numbers, as it is presented here through the simple and unadorned presentation of One and Zero or Nothing.

—Leibniz's letter to the Duke of Brunswick attached with the *I Ching* hexagrams^[33]

Later developments



George Boole

In 1854, British mathematician George Boole published a landmark paper detailing an algebraic system of logic that would become known as Boolean algebra. His logical calculus was to become instrumental in the design of digital electronic circuitry.^[35]

In 1937, Claude Shannon produced his master's thesis at MIT that implemented Boolean algebra and binary arithmetic using electronic relays and switches for the first time in history. Entitled *A Symbolic Analysis of Relay and Switching Circuits*, Shannon's thesis essentially founded practical digital circuit design.^[36]

In November 1937, George Stibitz, then working at Bell Labs, completed a relay-based computer he dubbed the "Model K" (for "**K**itchen", where he had assembled it), which calculated using binary addition.^[37] Bell Labs authorized a full research program in late 1938 with Stibitz at the helm. Their Complex Number Computer, completed 8 January 1940, was able to calculate complex numbers. In a demonstration to the American Mathematical Society conference at Dartmouth College on 11 September 1940, Stibitz was able to send the Complex Number Calculator remote commands over telephone lines by a teletype. It was the first computing machine ever used remotely over a phone line. Some participants of the conference who witnessed the demonstration were John von Neumann, John Mauchly and Norbert Wiener, who wrote about it in his memoirs.^{[38][39][40]}

The Z1 computer, which was designed and built by Konrad Zuse between 1935 and 1938, used Boolean logic and binary floating-point numbers.^[41]

Representation

Any number can be represented by a sequence of bits (binary digits), which in turn may be represented by any mechanism capable of being in two mutually exclusive states. Any of the following rows of symbols can be interpreted as the binary numeric value of 667:


```

1 0 1 0 0 1 1 0 1 1
| — | — — | | — | |
☒ ☐ ☒ ☐ ☐ ☒ ☒ ☐ ☒ ☒
y n y n n y y n y y
T F T F F T T F T T
+ - + - - + + - + +

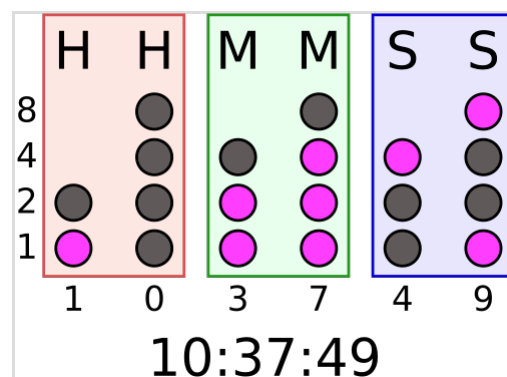
```

The numeric value represented in each case depends on the value assigned to each symbol. In the earlier days of computing, switches, punched holes, and punched paper tapes were used to represent binary values.^[42] In a modern computer, the numeric values may be represented by two different voltages; on a magnetic disk, magnetic polarities may be used. A "positive", "yes", or "on" state is not necessarily equivalent to the numerical value of one; it depends on the architecture in use.

In keeping with the customary representation of numerals using Arabic numerals, binary numbers are commonly written using the symbols **0** and **1**. When written, binary numerals are often subscripted, prefixed, or suffixed to indicate their base, or radix. The following notations are equivalent:

- 100101 binary (explicit statement of format)
- 100101b (a suffix indicating binary format; also known as Intel convention^{[43][44]})
- 100101B (a suffix indicating binary format)
- bin 100101 (a prefix indicating binary format)
- 100101₂ (a subscript indicating base-2 (binary) notation)
- %100101 (a prefix indicating binary format; also known as Motorola convention^{[43][44]})
- 0b100101 (a prefix indicating binary format, common in programming languages)
- 6b100101 (a prefix indicating number of bits in binary format, common in programming languages)
- #b100101 (a prefix indicating binary format, common in Lisp programming languages)

When spoken, binary numerals are usually read digit-by-digit, to distinguish them from decimal numerals. For example, the binary numeral 100 is pronounced *one zero zero*, rather than *one hundred*, to make its binary nature explicit and for purposes of correctness. Since the binary numeral 100 represents the value four, it would be confusing to refer to the numeral as *one hundred* (a word that represents a completely different value, or amount). Alternatively, the binary numeral 100 can be read out as "four" (the correct *value*), but this does not make its binary nature explicit.



A binary clock might use LEDs to express binary values. In this clock, each column of LEDs shows a binary-coded decimal numeral of the traditional sexagesimal time.

Counting in binary

Counting in binary is similar to counting in any other number system. Beginning with a single digit, counting proceeds through each symbol, in increasing order. Before examining binary counting, it is useful to briefly discuss the more familiar decimal counting system as a frame of reference.

Decimal counting

Decimal counting uses the ten symbols *0* through *9*. Counting begins with the incremental substitution of the least significant digit (rightmost digit) which is often called the *first digit*. When the available symbols for this position are exhausted, the least significant digit is reset to *0*, and the next digit of higher significance (one position to the left) is incremented (*overflow*), and incremental substitution of the low-order digit resumes. This method of reset and overflow is repeated for each digit of significance. Counting progresses as follows:

000, 001, 002, ... 007, 008, 009, (rightmost digit is reset to zero, and the digit to its left is incremented)

010, 011, 012, ...

...

090, 091, 092, ... 097, 098, 099, (rightmost two digits are reset to zeroes, and next digit is incremented)

100, 101, 102, ...

Binary counting

Binary counting follows the exact same procedure, and again the incremental substitution begins with the least significant binary digit, or *bit* (the rightmost one, also called the *first bit*), except that only the two symbols *0* and *1* are available. Thus, after a bit reaches 1 in binary, an increment resets it to 0 but also causes an increment of the next bit to the left:

0000,

0001, (rightmost bit starts over, and the next bit is incremented)

0010, 0011, (rightmost two bits start over, and the next bit is incremented)

0100, 0101, 0110, 0111, (rightmost three bits start over, and the next bit is incremented)

1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111 ...

In the binary system, each bit represents an increasing power of 2, with the rightmost bit representing 2^0 , the next representing 2^1 , then 2^2 , and so on. The value of a binary number is the sum of the powers of 2 represented by each "1" bit. For example, the binary number 100101 is converted to decimal form as follows:

2^4	2^3	2^2	2^1	2^0	
16	8	4	2	1	
0	0	0	0	0	00

This counter shows how to count in binary from numbers zero through thirty-one.

$$100101_2 = [(1) \times 2^5] + [(0) \times 2^4] + [(0) \times 2^3] + [(1) \times 2^2] + [(0) \times 2^1] + [(1) \times 2^0]$$

$$100101_2 = [1 \times 32] + [0 \times 16] + [0 \times 8] + [1 \times 4] + [0 \times 2] + [1 \times 1]$$

$$100101_2 = 37_{10}$$

Binary arithmetic

Arithmetic in binary is much like arithmetic in other numeral systems using positional notation. Addition, subtraction, multiplication, and division can be performed on binary numerals.

Addition

The simplest arithmetic operation in binary is addition. Adding two single-digit binary numbers is relatively simple, using a form of carrying:

$$\begin{aligned} 0 + 0 &\rightarrow 0 \\ 0 + 1 &\rightarrow 1 \\ 1 + 0 &\rightarrow 1 \\ 1 + 1 &\rightarrow 0, \text{ carry } 1 \text{ (since } 1 + 1 = 2 = 0 + (1 \times 2^1)) \end{aligned}$$

Adding two "1" digits produces a digit "0", while 1 will have to be added to the next column. This is similar to what happens in decimal when certain single-digit numbers are added together; if the result equals or exceeds the value of the radix (10), the digit to the left is incremented:

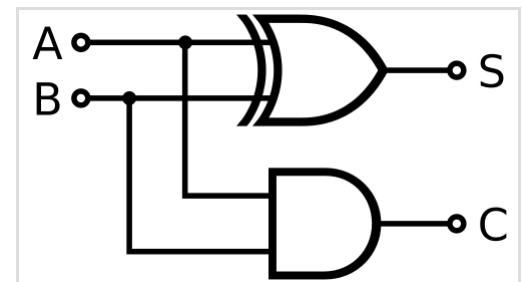
$$\begin{aligned} 5 + 5 &\rightarrow 0, \text{ carry } 1 \text{ (since } 5 + 5 = 10 = 0 + (1 \times 10^1)) \\ 7 + 9 &\rightarrow 6, \text{ carry } 1 \text{ (since } 7 + 9 = 16 = 6 + (1 \times 10^1)) \end{aligned}$$

This is known as *carrying*. When the result of an addition exceeds the value of a digit, the procedure is to "carry" the excess amount divided by the radix (that is, 10/10) to the left, adding it to the next positional value. This is correct since the next position has a weight that is higher by a factor equal to the radix. Carrying works the same way in binary:

1	1	1	1	1	(carried digits)		
0	1	1	0	1			
+	1	0	1	1			
=	1	0	0	1	0	0	= 36

1	3	5	7	+1+2+4
9	11	13	15	+8+16=
17	19	21	23	8 16
25	27	29	31	1 17
2	3	6	7	2 18
10	11	14	15	3 19
18	19	22	23	4 20
26	27	30	31	5 21
4	5	6	7	6 22
12	13	14	15	7 23
20	21	22	23	8 24
28	29	30	31	9 25
8	9	10	11	10 26
12	13	14	15	11 27
24	25	26	27	12 28
28	29	30	31	13 29
16	17	18	19	14 30
20	21	22	23	15 31
24	25	26	27	
28	29	30	31	

A party trick to guess a number from which cards it is printed on uses the bits of the binary representation of the number. In the SVG file, click a card to toggle it



The circuit diagram for a binary half adder, which adds two bits together, producing sum and carry bits

In this example, two numerals are being added together: 01101_2 (13_{10}) and 10111_2 (23_{10}). The top row shows the carry bits used. Starting in the rightmost column, $1 + 1 = 10_2$. The 1 is carried to the left, and the 0 is written at the bottom of the rightmost column. The second column from the right is added: $1 + 0 + 1 = 10_2$ again; the 1 is carried, and 0 is written at the bottom. The third column: $1 + 1 + 1 = 11_2$. This time, a 1 is carried, and a 1 is written in the bottom row. Proceeding like this gives the final answer 100100_2 (36_{10}).

When computers must add two numbers, the rule that: $x \text{ xor } y = (x + y) \bmod 2$ for any two bits x and y allows for very fast calculation, as well.

Long carry method

A simplification for many binary addition problems is the "long carry method" or "Brookhouse Method of Binary Addition". This method is particularly useful when one of the numbers contains a long stretch of ones. It is based on the simple premise that under the binary system, when given a stretch of digits composed entirely of n ones (where n is any integer length), adding 1 will result in the number 1 followed by a string of n zeros. That concept follows, logically, just as in the decimal system, where adding 1 to a string of n 9s will result in the number 1 followed by a string of n 0s:

Binary		likewise	Decimal
1 1 1 1 1			9 9 9 9 9
+ 1			+ 1
1 0 0 0 0 0			1 0 0 0 0 0

Such long strings are quite common in the binary system. From that one finds that large binary numbers can be added using two simple steps, without excessive carry operations. In the following example, two numerals are being added together: 111011110_2 (958_{10}) and 1010110011_2 (691_{10}), using the traditional carry method on the left, and the long carry method on the right:

Traditional Carry Method	vs.	Long Carry Method
$ \begin{array}{r} 111 \quad 11111 \\ 111011110 \\ + 1010110011 \\ \hline = 11001110001 \end{array} $	(carried digits) $1 \leftarrow 1 \leftarrow$	carry the 1 until it is one digit past the "string" below $ \begin{array}{r} \cancel{111}0\cancel{11111}0 \\ + 1010110011 \\ \hline 11001110001 \end{array} $ cross out the "string", and cross out the digit that was added

The top row shows the carry bits used. Instead of the standard carry from one column to the next, the lowest-ordered "1" with a "1" in the corresponding place value beneath it may be added and a "1" may be carried to one digit past the end of the series. The "used" numbers must be crossed off, since they are already added. Other long strings may likewise be cancelled using the same technique. Then, simply add together any remaining digits normally. Proceeding in this manner gives the final answer of 11001110001_2 (1649_{10}). In our simple example using small numbers, the traditional carry method required eight carry operations, yet the long carry method required only two, representing a substantial reduction of effort.

Addition table

	0	1
0	0	1
1	1	10

The binary addition table is similar to, but not the same as, the truth table of the logical disjunction operation $\vee$. The difference is that $1 \vee 1 = 1$, while $1 + 1 = 10$.

Subtraction

Subtraction works in much the same way:

$0 - 0 \rightarrow 0$
 $0 - 1 \rightarrow 1$, borrow 1
 $1 - 0 \rightarrow 1$
 $1 - 1 \rightarrow 0$

Subtracting a "1" digit from a "0" digit produces the digit "1", while 1 will have to be subtracted from the next column. This is known as *borrowing*. The principle is the same as for carrying. When the result of a subtraction is less than 0, the least possible value of a digit, the procedure is to "borrow" the deficit divided by the radix (that is, 10/10) from the left, subtracting it from the next positional value.

<pre> * * * * (starred columns are borrowed from) 1 1 0 1 1 1 0 - 1 0 1 1 1 ----- = 1 0 1 0 1 1 1 </pre>
<pre> * (starred columns are borrowed from) 1 0 1 1 1 1 1 - 1 0 1 0 1 1 ----- = 0 1 1 0 1 0 0 </pre>

Subtracting a positive number is equivalent to *adding* a negative number of equal absolute value. Computers use signed number representations to handle negative numbers—most commonly the two's complement notation. Such representations eliminate the need for a separate "subtract" operation. Using two's complement notation, subtraction can be summarized by the following formula:

$$A - B = A + \text{not } B + 1$$

Multiplication

Multiplication in binary is similar to its decimal counterpart. Two numbers A and B can be multiplied by partial products: for each digit in B , the product of that digit in A is calculated and

written on a new line, shifted leftward so that its rightmost digit lines up with the digit in B that was used. The sum of all these partial products gives the final result.

Since there are only two digits in binary, there are only two possible outcomes of each partial multiplication:

- If the digit in B is 0, the partial product is also 0
- If the digit in B is 1, the partial product is equal to A

For example, the binary numbers 1011 and 1010 are multiplied as follows:

$$\begin{array}{r}
 1011 \quad (A) \\
 \times 1010 \quad (B) \\
 \hline
 0000 \quad \leftarrow \text{to the rightmost 'zero' in } B \\
 + 1011 \quad \leftarrow \text{to the next 'one' in } B \\
 + 0000 \\
 + 1011 \\
 \hline
 = 1101110
 \end{array}$$

Binary numbers can also be multiplied with bits after a binary point:

$$\begin{array}{r}
 101.101 \quad A \text{ (5.625 in decimal)} \\
 \times 110.01 \quad B \text{ (6.25 in decimal)} \\
 \hline
 1.01101 \quad \leftarrow \text{to a 'one' in } B \\
 + 00.00000 \quad \leftarrow \text{to a 'zero' in } B \\
 + 000.0000 \\
 + 1011.01 \\
 + 10110.1 \\
 \hline
 = 100011.00101 \text{ (35.15625 in decimal)}
 \end{array}$$

See also Booth's multiplication algorithm.

Multiplication table

	0	1
0	0	0
1	0	1

The binary multiplication table is the same as the truth table of the logical conjunction operation $\wedge$.

Division

Long division in binary is again similar to its decimal counterpart.

In the example below, the divisor is 101_2 , or 5 in decimal, while the dividend is 11011_2 , or 27 in decimal. The procedure is the same as that of decimal long division; here, the divisor 101_2 goes

into the first three digits 110_2 of the dividend one time, so a "1" is written on the top line. This result is multiplied by the divisor, and subtracted from the first three digits of the dividend; the next digit (a "1") is included to obtain a new three-digit sequence:

$$\begin{array}{r}
 1 \\
 101 \overline{) 11011} \\
 \underline{- 101} \\
 001
 \end{array}$$

The procedure is then repeated with the new sequence, continuing until the digits in the dividend have been exhausted:

$$\begin{array}{r}
 101 \\
 101 \overline{) 11011} \\
 \underline{- 101} \\
 111 \\
 \underline{- 101} \\
 010
 \end{array}$$

Thus, the quotient of 11011_2 divided by 101_2 is 101_2 , as shown on the top line, while the remainder, shown on the bottom line, is 10_2 . In decimal, this corresponds to the fact that 27 divided by 5 is 5, with a remainder of 2.

Aside from long division, one can also devise the procedure so as to allow for over-subtracting from the partial remainder at each iteration, thereby leading to alternative methods which are less systematic, but more flexible as a result.

Square root

The process of taking a binary square root digit by digit is essentially the same as for a decimal square root but much simpler, due to the binary nature. First group the digits in pairs, using a leading 0 if necessary so there are an even number of digits. Now at each step, consider the answer so far, extended with the digits 01. If this can be subtracted from the current remainder, do so. Then extend the remainder with the next pair of digits. If you subtracted, the next digit of the answer is 1, otherwise it's 0.

$ \begin{array}{r} 1101 \\ \hline \sqrt{1010101} \\ \sqrt{1010101} \\ \hline -1 \\ \text{Answer so far is 0,} \\ \hline \text{extended by 01 is 001,} \\ 110 \\ \text{this CAN be subtracted} \\ -101 \end{array} $	$ \begin{array}{r} 1 \\ \hline \sqrt{1010101} \\ -1 \\ \hline 110 \end{array} $	$ \begin{array}{r} 11 \\ \hline \sqrt{1010101} \\ -1 \\ \hline 110 \\ -101 \end{array} $	$ \begin{array}{r} 110 \\ \hline \sqrt{1010101} \\ -1 \\ \hline 110 \\ -101 \end{array} $
--	--	--	---

from first pair 10, ----- so first digit of 1 10 01 answer is 1. - 1 10 01 ----- 0 Done! 1.	Answer so far is 1, extended by 01 is 101, this CAN be subtracted from remainder 110, so next answer digit is 1.	----- 1 10 Answer so far is 11, extended by 01 is 1101, this is TOO BIG to subtract from remainder 110, so next digit of answer is 0.	----- 1 10 01 Answer so far is 110, extended by 01 is 11001, this CAN be subtracted from remainder 11001, so next digit of answer is
--	--	--	--

Fractions

In binary arithmetic, the binary expansion of a fraction terminates only if the denominator is a power of 2. As a result, $1/10$ does not have a finite binary representation (10 has prime factors 2 and 5). This causes $10 \times 1/10$ not to precisely equal 1 in binary floating-point arithmetic. As an example, the binary expansion of $1/3$ is .010101..., which means that

$$\frac{1}{3} = 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} + \dots$$

An exact value cannot be found with a sum of a finite number of inverse powers of two, the zeros and ones in the binary representation of $1/3$ alternate forever.

Fraction	Decimal	Binary	Fractional approximation
1/1	1 or 0.999...	1 or $0.\overline{1}$	$1/2 + 1/4 + 1/8 \dots$
1/2	0.5 or 0.4999...	0.1 or $0.0\overline{1}$	$1/4 + 1/8 + 1/16 \dots$
1/3	0.333...	$0.0\overline{1}$	$1/4 + 1/16 + 1/64 \dots$
1/4	0.25 or 0.24999...	0.01 or $0.00\overline{1}$	$1/8 + 1/16 + 1/32 \dots$
1/5	0.2 or 0.1999...	$0.00\overline{11}$	$1/8 + 1/16 + 1/128 \dots$
1/6	0.1666...	$0.00\overline{1}$	$1/8 + 1/32 + 1/128 \dots$
1/7	0.142857142857...	$0.00\overline{1}$	$1/8 + 1/64 + 1/512 \dots$
1/8	0.125 or 0.124999...	0.001 or $0.000\overline{1}$	$1/16 + 1/32 + 1/64 \dots$
1/9	0.111...	$0.000\overline{111}$	$1/16 + 1/32 + 1/64 \dots$
1/10	0.1 or 0.0999...	$0.00\overline{011}$	$1/16 + 1/32 + 1/256 \dots$
1/11	0.090909...	$0.000\overline{1011101}$	$1/16 + 1/64 + 1/128 \dots$
1/12	0.08333...	$0.000\overline{1}$	$1/16 + 1/64 + 1/256 \dots$
1/13	0.076923076923...	$0.000\overline{100111011}$	$1/16 + 1/128 + 1/256 \dots$
1/14	0.0714285714285...	$0.000\overline{1}$	$1/16 + 1/128 + 1/1024 \dots$
1/15	0.0666...	$0.000\overline{1}$	$1/16 + 1/256 \dots$
1/16	0.0625 or 0.0624999...	0.0001 or $0.0000\overline{1}$	$1/32 + 1/64 + 1/128 \dots$

Bitwise operations

Though not directly related to the numerical interpretation of binary symbols, sequences of bits may be manipulated using Boolean logical operators. When a string of binary symbols is manipulated in this way, it is called a bitwise operation; the logical operators AND, OR, and XOR may be performed on corresponding bits in two binary numerals provided as input. The logical NOT operation may be performed on individual bits in a single binary numeral provided as input. Sometimes, such operations may be used as arithmetic short-cuts, and may have other computational benefits as well. For example, an arithmetic shift left of a binary number is the equivalent of multiplication by a (positive, integral) power of 2.

Conversion to and from other numeral systems

Decimal to binary

To convert from a base-10 integer to its base-2 (binary) equivalent, the number is divided by two. The remainder is the least-significant bit. The quotient is again divided by two; its remainder

becomes the next least significant bit. This process repeats until a quotient of one is reached. The sequence of remainders (including the final quotient of one) forms the binary value, as each remainder must be either zero or one when dividing by two. For example, $(357)_{10}$ is expressed as $(101100101)_2$.^[45]

Binary to decimal

Conversion from base-2 to base-10 simply inverts the preceding algorithm. The bits of the binary number are used one by one, starting with the most significant (leftmost) bit. Beginning with the value 0, the prior value is doubled, and the next bit is then added to produce the next value. This can be organized in a multi-column table. For example, to convert 10010101101_2 to decimal:

$$\begin{array}{r} 2 \overline{)357} \end{array}$$

Conversion of $(357)_{10}$ to binary notation results in (101100101)

Prior value	$\times 2 +$	Next bit	= Next value
	$0 \times 2 +$	1	= 1
	$1 \times 2 +$	0	= 2
	$2 \times 2 +$	0	= 4
	$4 \times 2 +$	1	= 9
	$9 \times 2 +$	0	= 18
	$18 \times 2 +$	1	= 37
	$37 \times 2 +$	0	= 74
	$74 \times 2 +$	1	= 149
	$149 \times 2 +$	1	= 299
	$299 \times 2 +$	0	= 598
	$598 \times 2 +$	1	= 1197

The result is 1197_{10} . The first Prior Value of 0 is simply an initial decimal value. This method is an application of the Horner scheme.

Binary	1	0	0	1	0	1	0	1	1	0	1	
Decimal	1×2^{10}	0×2^9	0×2^8	1×2^7	0×2^6	1×2^5	0×2^4	1×2^3	1×2^2	0×2^1	1×2^0	1197
	+	+	+	+	+	+	+	+	+	+	=	

The fractional parts of a number are converted with similar methods. They are again based on the equivalence of shifting with doubling or halving.

In a fractional binary number such as 0.11010110101_2 , the first digit is $\frac{1}{2}$, the second $(\frac{1}{2})^2 = \frac{1}{4}$, etc. So if there is a 1 in the first place after the decimal, then the number is at least $\frac{1}{2}$, and vice versa. Double that number is at least 1. This suggests the algorithm: Repeatedly double the number to be converted, record if the result is at least 1, and then throw away the integer part.

For example, $(\frac{1}{3})_{10}$, in binary, is:

Converting	Result
$\frac{1}{3}$	0.
$\frac{1}{3} \times 2 = \frac{2}{3} < 1$	0.0
$\frac{2}{3} \times 2 = 1\frac{1}{3} \geq 1$	0.01
$\frac{1}{3} \times 2 = \frac{2}{3} < 1$	0.010
$\frac{2}{3} \times 2 = 1\frac{1}{3} \geq 1$	0.0101

Thus the repeating decimal fraction $0.\overline{3}...$ is equivalent to the repeating binary fraction $0.\overline{01}...$.

Or for example, 0.1_{10} , in binary, is:

Converting	Result
0.1	0.
$0.1 \times 2 = \mathbf{0.2} < 1$	0.0
$0.2 \times 2 = \mathbf{0.4} < 1$	0.00
$0.4 \times 2 = \mathbf{0.8} < 1$	0.000
$0.8 \times 2 = \mathbf{1.6} \geq 1$	0.0001
$0.6 \times 2 = \mathbf{1.2} \geq 1$	0.00011
$0.2 \times 2 = \mathbf{0.4} < 1$	0.000110
$0.4 \times 2 = \mathbf{0.8} < 1$	0.0001100
$0.8 \times 2 = \mathbf{1.6} \geq 1$	0.00011001
$0.6 \times 2 = \mathbf{1.2} \geq 1$	0.000110011
$0.2 \times 2 = \mathbf{0.4} < 1$	0.0001100110

This is also a repeating binary fraction $0.\overline{00011}...$. It may come as a surprise that terminating decimal fractions can have repeating expansions in binary. It is for this reason that many are surprised to discover that $1/10 + ... + 1/10$ (addition of 10 numbers) differs from 1 in binary floating-point arithmetic. In fact, the only binary fractions with terminating expansions are of the form of an integer divided by a power of 2, which $1/10$ is not.

The final conversion is from binary to decimal fractions. The only difficulty arises with repeating fractions, but otherwise the method is to shift the fraction to an integer, convert it as above, and then divide by the appropriate power of two in the decimal base. For example:

$$x = 1100.\overline{101110}...$$

$$\begin{aligned}
 \overline{x} \times 2^6 &= 1100101110.\overline{01110} \dots \\
 \overline{x} \times 2 &= 11001.\overline{01110} \dots \\
 x \times (2^6 - 2) &= 1100010101 \\
 x &= 1100010101/111110 \\
 x &= (789/62)_{10}
 \end{aligned}$$

Another way of converting from binary to decimal, often quicker for a person familiar with hexadecimal, is to do so indirectly—first converting (x in binary) into (x in hexadecimal) and then converting (x in hexadecimal) into (x in decimal).

For very large numbers, these simple methods are inefficient because they perform a large number of multiplications or divisions where one operand is very large. A simple divide-and-conquer algorithm is more effective asymptotically: given a binary number, it is divided by 10^k , where k is chosen so that the quotient roughly equals the remainder; then each of these pieces is converted to decimal and the two are concatenated. Given a decimal number, it can be split into two pieces of about the same size, each of which is converted to binary, whereupon the first converted piece is multiplied by 10^k and added to the second converted piece, where k is the number of decimal digits in the second, least-significant piece before conversion.

Hexadecimal

Binary may be converted to and from hexadecimal more easily. This is because the radix of the hexadecimal system (16) is a power of the radix of the binary system (2). More specifically, $16 = 2^4$, so it takes four digits of binary to represent one digit of hexadecimal, as shown in the adjacent table.

To convert a hexadecimal number into its binary equivalent, simply substitute the corresponding binary digits:

$$\begin{aligned}
 3A_{16} &= 0011\ 1010_2 \\
 E7_{16} &= 1110\ 0111_2
 \end{aligned}$$

To convert a binary number into its hexadecimal equivalent, divide it into groups of four bits. If the number of bits is not a multiple of four, simply insert extra **0** bits at the left (called padding). For example:

$$\begin{aligned}
 1010010_2 &= 0101\ 0010 \text{ grouped with padding} = 52_{16} \\
 11011101_2 &= 1101\ 1101 \text{ grouped} = DD_{16}
 \end{aligned}$$

0 _{hex} = <u>0</u> _{dec} = <u>0</u> _{oct}	0	0	0	0
1 _{hex} = <u>1</u> _{dec} = <u>1</u> _{oct}	0	0	0	1
2 _{hex} = <u>2</u> _{dec} = <u>2</u> _{oct}	0	0	1	0
3 _{hex} = <u>3</u> _{dec} = <u>3</u> _{oct}	0	0	1	1
4 _{hex} = <u>4</u> _{dec} = <u>4</u> _{oct}	0	1	0	0
5 _{hex} = <u>5</u> _{dec} = <u>5</u> _{oct}	0	1	0	1
6 _{hex} = <u>6</u> _{dec} = <u>6</u> _{oct}	0	1	1	0
7 _{hex} = <u>7</u> _{dec} = <u>7</u> _{oct}	0	1	1	1
8 _{hex} = <u>8</u> _{dec} = <u>10</u> _{oct}	1	0	0	0
9 _{hex} = <u>9</u> _{dec} = <u>11</u> _{oct}	1	0	0	1
A _{hex} = <u>10</u> _{dec} = <u>12</u> _{oct}	1	0	1	0
B _{hex} = <u>11</u> _{dec} = <u>13</u> _{oct}	1	0	1	1
C _{hex} = <u>12</u> _{dec} = <u>14</u> _{oct}	1	1	0	0
D _{hex} = <u>13</u> _{dec} = <u>15</u> _{oct}	1	1	0	1
E _{hex} = <u>14</u> _{dec} = <u>16</u> _{oct}	1	1	1	0
F _{hex} = <u>15</u> _{dec} = <u>17</u> _{oct}	1	1	1	1

To convert a hexadecimal number into its decimal equivalent, multiply the decimal equivalent of

each hexadecimal digit by the corresponding power of 16 and add the resulting values:

$$\text{C0E7}_{16} = (12 \times 16^3) + (0 \times 16^2) + (14 \times 16^1) + (7 \times 16^0) = (12 \times 4096) + (0 \times 256) + (14 \times 16) + (7 \times 1) = 49,383_{10}$$

Octal

Binary is also easily converted to the octal numeral system, since octal uses a radix of 8, which is a power of two (namely, 2^3 , so it takes exactly three binary digits to represent an octal digit). The correspondence between octal and binary numerals is the same as for the first eight digits of hexadecimal in the table above. Binary 000 is equivalent to the octal digit 0, binary 111 is equivalent to octal 7, and so forth.

Octal	Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Converting from octal to binary proceeds in the same fashion as it does for hexadecimal:

$$65_8 = 110\ 101_2$$

$$17_8 = 001\ 111_2$$

And from binary to octal:

$$101100_2 = 101\ 100_2 \text{ grouped} = 54_8$$

$$10011_2 = 010\ 011_2 \text{ grouped with padding} = 23_8$$

And from octal to decimal:

$$65_8 = (6 \times 8^1) + (5 \times 8^0) = (6 \times 8) + (5 \times 1) = 53_{10}$$

$$127_8 = (1 \times 8^2) + (2 \times 8^1) + (7 \times 8^0) = (1 \times 64) + (2 \times 8) + (7 \times 1) = 87_{10}$$

Representing real numbers

Non-integers can be represented by using negative powers, which are set off from the other digits by means of a radix point (called a decimal point in the decimal system). For example, the

binary number 11.01_2 means:

$$1 \times 2^1 \quad (1 \times 2 = 2) \quad \text{plus}$$

$$1 \times 2^0 \quad (1 \times 1 = 1) \quad \text{plus}$$

$$0 \times 2^{-1} \quad (0 \times \tfrac{1}{2} = 0) \quad \text{plus}$$

$$1 \times 2^{-2} \quad (1 \times \tfrac{1}{4} = 0.25)$$

For a total of 3.25 decimal.

All dyadic rational numbers $\frac{p}{2^a}$ have a *terminating* binary numeral—the binary representation has a finite number of terms after the radix point. Other rational numbers have binary representation, but instead of terminating, they *recur*, with a finite sequence of digits repeating indefinitely. For instance

$$\frac{1_{10}}{3_{10}} = \frac{1_2}{11_2} = 0.01010101\overline{01} \dots_2$$

$$\frac{12_{10}}{17_{10}} = \frac{1100_2}{10001_2} = 0.1011010010110100\overline{10110100} \dots_2$$

The phenomenon that the binary representation of any rational is either terminating or recurring also occurs in other radix-based numeral systems. See, for instance, the explanation in decimal. Another similarity is the existence of alternative representations for any terminating representation, relying on the fact that 0.111111... is the sum of the geometric series $2^{-1} + 2^{-2} + 2^{-3} + \dots$ which is 1.

Binary numerals that neither terminate nor recur represent irrational numbers. For instance,

- 0.10100100010000100000100... does have a pattern, but it is not a fixed-length recurring pattern, so the number is irrational
- 1.0110101000001001111001100110011111110... is the binary representation of $\sqrt{2}$, the square root of 2, another irrational. It has no discernible pattern.

See also

- ASCII
- Balanced ternary
- Bitwise operation
- Binary code
- Binary-coded decimal
- Binary non positional code
- Finger binary



- [Gray code](#)
- [IEEE 754](#)
- [Linear-feedback shift register](#)
- [Offset binary](#)
- [Quibinary](#)
- [Reduction of summands](#)
- [Redundant binary representation](#)
- [Repeating decimal](#)
- [Two's complement](#)
- [Unicode](#)

References

1. "3.3. Binary and Its Advantages — CS160 Reader" (<https://computerscience.chemeketa.edu/cs160Reader/Binary/Binary.html>). *computerscience.chemeketa.edu*. Retrieved 22 May 2024.
2. "Why Did Thomas Harriot Invent Binary?" (<https://link.springer.com/article/10.1007/s00283-023-10271-9>). *springer.com*. Retrieved 17 April 2026.
3. "History of The Binary Number System" (<https://www.convertbinary.com/blog/binary-number-system-history/>). *convertbinary.com*. Retrieved 17 April 2026.
4. Robson, Eleanor; Stedall, Jacqueline, eds. (2009), "Myth No. 2: the Horus eye fractions", *The Oxford Handbook of the History of Mathematics* (<https://books.google.com/books?id=xZMSDAAQBAJ&pg=PA790>), Oxford University Press, p. 790, ISBN 9780199213122
5. Chrisomalis, Stephen (2010), *Numerical Notation: A Comparative History* (<https://books.google.com/books?id=ux--OWgWvBQC&pg=PA42>), Cambridge University Press, pp. 42–43, ISBN 9780521878180.
6. Rudman, Peter Strom (2007), *How Mathematics Happened: The First 50,000 Years* (<https://books.google.com/books?id=BtcQq4RUfkUC&pg=PA135>), Prometheus Books, pp. 135–136, ISBN 9781615921768.
7. Edward Hacker; Steve Moore; Lorraine Patsco (2002). *I Ching: An Annotated Bibliography* (<https://books.google.com/books?id=S5hLpfFiMCQC&pg=PR13>). Routledge. p. 13. ISBN 978-0-415-93969-0.
8. Redmond, Geoffrey; Hon, Tze-Ki (2014). *Teaching the I Ching*. Oxford University Press. p. 227. ISBN 978-0-19-976681-9.
9. Jonathan Shectman (2003). *Groundbreaking Scientific Experiments, Inventions, and Discoveries of the 18th Century* (<https://books.google.com/books?id=SsbChdIflsC&pg=PA29>). Greenwood Publishing. p. 29. ISBN 978-0-313-32015-6.
10. Marshall, Steve. "Yijing hexagram sequences: The Shao Yong square (Fuxi sequence)" (<http://www.biroco.com/yijing/sequence.htm>). Retrieved 15 September 2022. "You could say [the Fuxi binary sequence] is a more sensible way of rendering hexagram as binary numbers The reasoning, if any, that informs [the King Wen] sequence is unknown."

11. Zhonglian, Shi; Wenzhao, Li; Poser, Hans (2000). *Leibniz' Binary System and Shao Yong's "Xiantian Tu" in :Das Neueste über China: G.W. Leibnizens Novissima Sinica von 1697 : Internationales Symposium, Berlin 4. bis 7. Oktober 1997* (<https://books.google.com/books?id=DkIpP2SsGIIC&pg=PA165>). Stuttgart: Franz Steiner Verlag. pp. 165–170. ISBN 3515074481.
12. Collins, Derek (2008). "Mapping the Entrails: The Practice of Greek Hepatoscopy" (<https://www.jstor.org/stable/27566714>). *The American Journal of Philology*. **129** (3): 319–345. ISSN 0002-9475 (<https://search.worldcat.org/issn/0002-9475>). JSTOR 27566714 (<https://www.jstor.org/stable/27566714>).
13. Johnston, Sarah Iles (2008). *Ancient Greek divination*. Blackwell ancient religions (1. publ ed.). Malden, Mass.: Wiley-Blackwell. ISBN 978-1-4051-1573-5.
14. Sanchez, Julio; Canton, Maria P. (2007). *Microcontroller programming: the microchip PIC*. Boca Raton, Florida: CRC Press. p. 37. ISBN 978-0-8493-7189-9.
15. W. S. Anglin and J. Lambek, *The Heritage of Thales*, Springer, 1995, ISBN 0-387-94544-X
16. Math for Poets and Drummers (<http://www.sju.edu/~rhall/Rhythms/Poets/arcadia.pdf>) Archived (<https://web.archive.org/web/20120616225617/http://www.sju.edu/~rhall/Rhythms/Poets/arcadia.pdf>) 16 June 2012 at the Wayback Machine (pdf, 145KB)
17. Stakhov, Alexey; Olsen, Scott Anthony (2009). *The mathematics of harmony: from Euclid to contemporary mathematics and computer science* (<https://books.google.com/books?id=K6fac9RxXREC>). World Scientific. ISBN 978-981-277-582-5.
18. B. van Nooten, "Binary Numbers in Indian Antiquity", *Journal of Indian Studies*, Volume 21, 1993, pp. 31–50
19. Landry (2019), p. 25.
20. Landry (2019), p. 154.
21. "Ifa Divination System" (<https://ich.unesco.org/en/RL/ifa-divination-system-00146>). Retrieved 5 July 2017.
22. Bender, Andrea; Beller, Sieghard (16 December 2013). "Mangarevan invention of binary steps for easier calculation" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3910603>). *Proceedings of the National Academy of Sciences*. **111** (4): 1322–1327. doi:10.1073/pnas.1309160110 (<https://doi.org/10.1073%2Fpnas.1309160110>). PMC 3910603 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3910603>). PMID 24344278 (<https://pubmed.ncbi.nlm.nih.gov/24344278>).
23. Bowern, Claire; Zentz, Jason (2012). "Diversity in the Numeral Systems of Australian Languages" (<https://www.jstor.org/stable/23621076>). *Anthropological Linguistics*. **54** (2): 133–160. ISSN 0003-5483 (<https://search.worldcat.org/issn/0003-5483>). JSTOR 23621076 (<https://www.jstor.org/stable/23621076>).
24. (see Bonner 2007 [1] (<http://lullianarts.net/>) Archived (<https://web.archive.org/web/20140403194204/http://lullianarts.net/>) 3 April 2014 at the Wayback Machine, Fidora et al. 2011 [2] (<https://www.iiia.csic.es/es/publications/ramon-llull-ars-magna-artificial-intelligence/>) Archived (<https://web.archive.org/web/20190408011909/https://www.iiia.csic.es/es/publications/ramon-llull-ars-magna-artificial-intelligence/>) 8 April 2019 at the Wayback Machine)

25. Bacon, Francis (1605). "The Advancement of Learning" (<http://home.hiwaay.net/~paul/bacon/advancement/book6ch1.html>). London. pp. Chapter 1.
26. Shirley, John W. (1951). "Binary numeration before Leibniz". *American Journal of Physics*. **19** (8): 452–454. Bibcode:1951AmJPh..19..452S (<https://ui.adsabs.harvard.edu/abs/1951AmJPh..19..452S>). doi:10.1119/1.1933042 (<https://doi.org/10.1119%2F1.1933042>).
27. Ineichen, R. (2008). "Leibniz, Caramuel, Harriot und das Dualsystem" (<https://web.archive.org/web/20211228024010/http://page.math.tu-berlin.de/~mdmv/archive/16/mdmv-16-1-12-ineichen.pdf>) (PDF). *Mitteilungen der deutschen Mathematiker-Vereinigung* (in German). **16** (1): 12–15. doi:10.1515/dmvm-2008-0009 (<https://doi.org/10.1515%2Fdmvm-2008-0009>). S2CID 179000299 (<https://api.semanticscholar.org/CorpusID:179000299>). Archived from the original (<http://page.math.tu-berlin.de/~mdmv/archive/16/mdmv-16-1-12-ineichen.pdf>) (PDF) on 28 December 2021. Retrieved 15 October 2017.
28. Strickland, Lloyd (2020), Sriraman, Bharath (ed.), "Leibniz on Number Systems" (https://link.springer.com/referenceworkentry/10.1007/978-3-030-19071-2_90-1), *Handbook of the History and Philosophy of Mathematical Practice*, Cham: Springer International Publishing, pp. 1–31, doi:10.1007/978-3-030-19071-2_90-1 (https://doi.org/10.1007%2F978-3-030-19071-2_90-1), ISBN 978-3-030-19071-2, retrieved 20 August 2024
29. Leibniz G., Explication de l'Arithmétique Binaire, Die Mathematische Schriften, ed. C. Gerhardt, Berlin 1879, vol.7, p.223; Engl. transl.[3] (<https://www.leibniz-translations.com/binary>)
30. "Bouvet and Leibniz: A Scholarly Correspondence" (<https://gwern.net/doc/cs/1980-swiderski.pdf#page=8>), Swiderski 1980
31. Leibniz (<https://www.leibniz-translations.com/binary.htm>): "The Chinese lost the meaning of the Cova or Lineations of Fuxi, perhaps more than a thousand years ago, and they have written commentaries on the subject in which they have sought I know not what far out meanings, so that their true explanation now has to come from Europeans. Here is how: It was scarcely more than two years ago that I sent to Reverend Father Bouvet,³ the celebrated French Jesuit who lives in Peking, my method of counting by 0 and 1, and nothing more was required to make him recognize that this was the key to the figures of Fuxi. Writing to me on 14 November 1701, he sent me this philosophical prince's grand figure, which goes up to 64, and leaves no further room to doubt the truth of our interpretation, such that it can be said that this Father has deciphered the enigma of Fuxi, with the help of what I had communicated to him. And as these figures are perhaps the most ancient monument of [GM VII, p227] science which exists in the world, this restitution of their meaning, after such a great interval of time, will seem all the more curious."
32. Aiton, Eric J. (1985). *Leibniz: A Biography*. Taylor & Francis. pp. 245–8. ISBN 0-85274-470-6.
33. J.E.H. Smith (2008). *Leibniz: What Kind of Rationalist?: What Kind of Rationalist?* (https://books.google.com/books?id=Da_oP3sJs1oC&pg=PA4153). Springer. p. 415. ISBN 978-1-4020-8668-7.
34. Yuen-Ting Lai (1998). *Leibniz, Mysticism and Religion* (<https://books.google.com/books?id=U9dOmVt81UAC&pg=PA149>). Springer. pp. 149–150. ISBN 978-0-7923-5223-5.

35. Boole, George (2009) [1854]. *An Investigation of the Laws of Thought on Which are Founded the Mathematical Theories of Logic and Probabilities* (<https://www.gutenberg.org/ebooks/15114>) (Macmillan, Dover Publications, reprinted with corrections [1958] ed.). New York: Cambridge University Press. ISBN 978-1-108-00153-3.
36. Shannon, Claude Elwood (1940). *A symbolic analysis of relay and switching circuits* (Thesis). Cambridge: Massachusetts Institute of Technology. [hdl:1721.1/11173](https://hdl.handle.net/1721.1/11173) (<https://hdl.handle.net/1721.1/11173>).
37. "National Inventors Hall of Fame – George R. Stibitz" (https://web.archive.org/web/20100709213530/http://www.invent.org/hall_of_fame/140.html). 20 August 2008. Archived from the original (http://www.invent.org/hall_of_fame/140.html) on 9 July 2010. Retrieved 5 July 2010.
38. "George Stibitz : Bio" (<http://stibitz.denison.edu/bio.html>). Math & Computer Science Department, Denison University. 30 April 2004. Retrieved 5 July 2010.
39. "Pioneers – The people and ideas that made a difference – George Stibitz (1904–1995)" (<http://www.kerryr.net/pioneers/stibitz.htm>). Kerry Redshaw. 20 February 2006. Retrieved 5 July 2010.
40. "George Robert Stibitz – Obituary" (<http://ei.cs.vt.edu/~history/Stibitz.html>). Computer History Association of California. 6 February 1995. Retrieved 5 July 2010.
41. Rojas, Raúl (April–June 1997). "Konrad Zuse's Legacy: The Architecture of the Z1 and Z3" (http://ed-thelen.org/comp-hist/Zuse_Z1_and_Z3.pdf) (PDF). *IEEE Annals of the History of Computing*. **19** (2): 5–16. doi:10.1109/85.586067 (<https://doi.org/10.1109/85.586067>). Archived (https://web.archive.org/web/20220703082408/http://ed-thelen.org/comp-hist/Zuse_Z1_and_Z3.pdf) (PDF) from the original on 3 July 2022. Retrieved 3 July 2022. (12 pages)
42. "Introducing binary – Revision 1 – GCSE Computer Science" (<https://web.archive.org/web/20190626050826/https://www.bbc.com/bitesize/guides/zwsbwmn/revision/1>). BBC. Archived from the original (<https://www.bbc.com/bitesize/guides/zwsbwmn/revision/1>) on 26 June 2019. Retrieved 26 June 2019.
43. Küveler, Gerd; Schwach, Dietrich (2013) [1996]. *Arbeitsbuch Informatik – eine praxisorientierte Einführung in die Datenverarbeitung mit Projektaufgabe* (<https://books.google.com/books?id=b8-dBgAAQBAJ>) (in German). Vieweg-Verlag, reprint: Springer-Verlag. doi:10.1007/978-3-322-92907-5 (<https://doi.org/10.1007/978-3-322-92907-5>). ISBN 978-3-528-04952-2. 9783322929075.
44. Küveler, Gerd; Schwach, Dietrich (4 October 2007). *Informatik für Ingenieure und Naturwissenschaftler: PC- und Mikrocomputertechnik, Rechnernetze* (<https://books.google.com/books?id=xQbvPYxceY0C>) (in German). Vol. 2 (5 ed.). Vieweg, reprint: Springer-Verlag. ISBN 978-3834891914. 9783834891914.
45. "Base System" (<https://web.archive.org/web/20171023010521/https://www.chalkstreet.com/aptipedia/knowledgebase/base-system/>). Archived from the original on 23 October 2017. Retrieved 31 August 2016.

Sources

Landry, Timothy R. (2019). *Vodún: Secrecy and the Search for Divine Power*. Contemporary ethnography (1st ed.). Philadelphia: University of Pennsylvania Press.
ISBN 978-0-8122-5074-9.

External links

- [Binary System \(http://www.cut-the-knot.org/do_you_know/BinaryHistory.shtml\)](http://www.cut-the-knot.org/do_you_know/BinaryHistory.shtml) at [cut-the-knot](http://www.cut-the-knot.org)
 - [Conversion of Fractions \(http://www.cut-the-knot.org/blue/frac_conv.shtml\)](http://www.cut-the-knot.org/blue/frac_conv.shtml) at [cut-the-knot](http://www.cut-the-knot.org)
 - [Sir Francis Bacon's BiLiteral Cypher system Archived \(https://web.archive.org/web/20160923014940/http://www.baconlinks.com/docs/BILITERAL.doc\)](https://web.archive.org/web/20160923014940/http://www.baconlinks.com/docs/BILITERAL.doc) 23 September 2016 at the Wayback Machine, predates binary number system.
 - [An analysis of binary as an efficient base \(https://www.youtube.com/watch?v=rDDaEVcwIJM\)](https://www.youtube.com/watch?v=rDDaEVcwIJM)
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=Binary_number&oldid=1352439387"